

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



“P04 Gestión de hilos”

Asignatura:

Sistemas Operativos

Docente:

Ing. Hugo Manuel Barraza Vizcarra

Estudiante:

Grecia Irene Meneses Chaupi

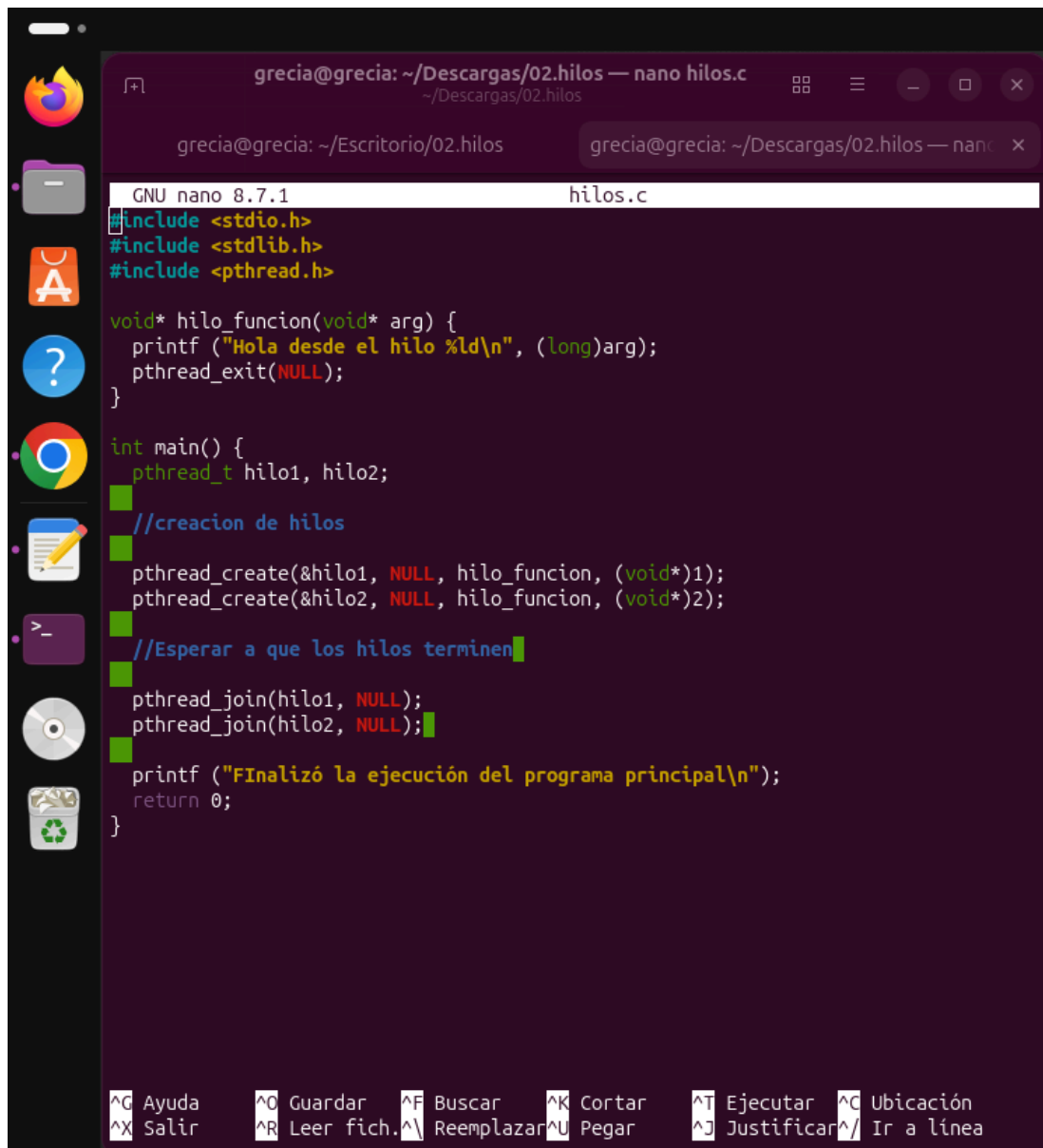
2021-119055

Tacna - Perú

2026

1. Procedimiento

1.1. Creación y gestión de hilos en C



```
greCIA@grecia: ~/Descargas/02.hilos — nano hilos.c
greCIA@grecia: ~/Escritorio/02.hilos
GNU nano 8.7.1 hilos.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* hilo_funcion(void* arg) {
    printf ("Hola desde el hilo %ld\n", (long)arg);
    pthread_exit(NULL);
}

int main() {
    pthread_t hilo1, hilo2;

    //creacion de hilos

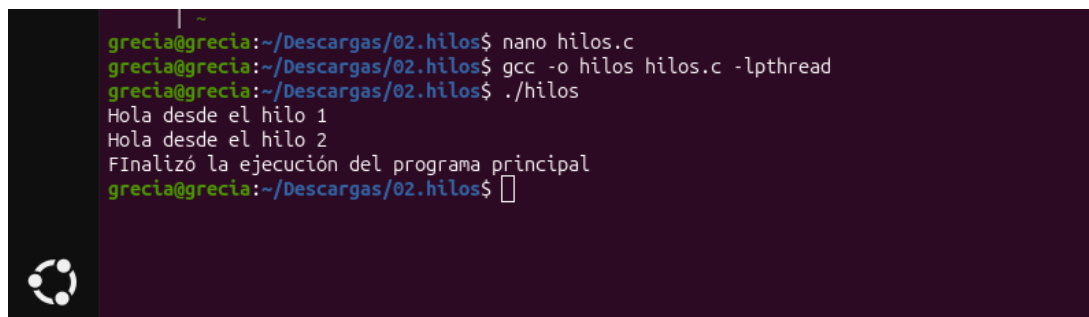
    pthread_create(&hilo1, NULL, hilo_funcion, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion, (void*)2);

    //Esperar a que los hilos terminen

    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    printf ("Finalizó la ejecución del programa principal\n");
    return 0;
}
```

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^_ Ir a línea



```
greCIA@grecia:~/Descargas/02.hilos$ nano hilos.c
greCIA@grecia:~/Descargas/02.hilos$ gcc -o hilos hilos.c -lpthread
greCIA@grecia:~/Descargas/02.hilos$ ./hilos
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal
greCIA@grecia:~/Descargas/02.hilos$
```

1.2. Implementación de sincronización

```
greCIA@grecia: ~/Descargas/02.hilos — nano hilos.c
~/Descargas/02.hilos
greCIA@grecia: ~/Escritorio/02.hilos
greCIA@grecia: ~/Descargas/02.hilos — nano hilos.c

GNU nano 8.7.1 hilos.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t mutex;
void* hilo_funcion_con_mutex (void* arg) {
    pthread_mutex_lock (&mutex); // Bloquear el mutex
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_mutex_unlock (&mutex); // Desbloquear el mutex
    pthread_exit(NULL);
}

int main() {
    pthread_t hilo1, hilo2;

    pthread_mutex_init(&mutex, NULL);
    //creacion de hilos

    pthread_create(&hilo1, NULL, hilo_funcion_con_mutex, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion_con_mutex, (void*)2);

    //Esperar a que los hilos terminen

    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    pthread_mutex_destroy(&mutex);

    printf ("Finalizó la ejecución del programa principal\n");
    return 0;
}

[ 31 líneas leídas ]
^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^_ Ir a línea
```

```
greCIA@grecia: ~/Descargas/02.hilos
~/Descargas/02.hilos

hilos.c: In function 'main':
hilos.c:27:9: error: implicit declaration of function 'pthread_mutex_destroy'; did
you mean 'pthread_mutex_destroy'? [-Wimplicit-function-declaration]
27 |         pthread_mutex_destroy(&mutex);
   |         ^~~~~~
   |         pthread_mutex_destroy
greCIA@grecia:~/Descargas/02.hilos$ nano hilos.c
greCIA@grecia:~/Descargas/02.hilos$ gcc -o hilos hilos.c -lpthread
greCIA@grecia:~/Descargas/02.hilos$ ./hilos
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal
greCIA@grecia:~/Descargas/02.hilos$
```

1.3. Análisis del rendimiento

```
grecia@grecia: ~/Descargas/02.hilos — nano hilos.c
GNU nano 8.7.1 hilos.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

pthread_mutex_t mutex; // Declaración del mutex

// Función del hilo
void* hilo_funcion(void* arg) {
    pthread_mutex_lock(&mutex); // Bloquear el mutex
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_mutex_unlock(&mutex); // Desbloquear el mutex
    pthread_exit(NULL);
}

void ejecutar_con_hilos(int num_hilos) {
    pthread_t hilos[num_hilos];

    // Inicializar el mutex
    pthread_mutex_init(&mutex, NULL);

    // Medición de tiempo
    clock_t start, end;
    start = clock(); // Inicio del temporizador

    // Crear hilos
    for (long i = 0; i < num_hilos; i++) {
        pthread_create(&hilos[i], NULL, hilo_funcion, (void*)i);
    }

    // Esperar a que los hilos terminen
    for (int i = 0; i < num_hilos; i++) {
        pthread_join(hilos[i], NULL);
    }

    end = clock(); // Fin del temporizador

    // Destruir el mutex
    pthread_mutex_destroy(&mutex);

    // Calcular y mostrar el tiempo total de ejecución
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    [ 60 líneas leídas ]

```

^G Ayuda	^O Guardar	^F Buscar	^K Cortar	^T Ejecutar	^G Ubicación	M-U Deshacer
^X Salir	^R Leer fich.	^I Reemplazar	^U Pegar	^J Justificar	^_ Ir a línea	M-E Rehacer

```
greCIA@greCIA: ~/Descargas/02.hilos — nano hilos.c
~/Descargas/02.hilos

GNU nano 8.7.1 hilos.c
pthread_mutex_init(&mutex, NULL);

// Medición de tiempo
clock_t start, end;
start = clock(); // Inicio del temporizador

// Crear hilos
for (long i = 0; i < num_hilos; i++) {
    pthread_create(&hilos[i], NULL, hilo_funcion, (void*)i);
}

// Esperar a que los hilos terminen
for (int i = 0; i < num_hilos; i++) {
    pthread_join(hilos[i], NULL);
}

end = clock(); // Fin del temporizador

// Destruir el mutex
pthread_mutex_destroy(&mutex);

// Calcular y mostrar el tiempo total de ejecución
double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("Tiempo total de ejecución con %d hilos: %f segundos\n",
       num_hilos, cpu_time_used);
}

int main() {
    printf("Ejecutando con 1 hilo...\n");
    ejecutar_con_hilos(1);

    printf("Ejecutando con 5 hilos...\n");
    ejecutar_con_hilos(5);

    printf("Ejecutando con 10 hilos...\n");
    ejecutar_con_hilos(10);

    printf("Finalizó la ejecución del programa principal\n");
    return 0;
}

^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación  M-U Deshacer
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^_ Ir a línea  M-E Rehacer
```

```

greCIA@grecia:~/Descargas/02.hilos$ gcc -o hilos hilos.c -lpthread
greCIA@grecia:~/Descargas/02.hilos$ ./hilos
Ejecutando con 1 hilo...
Hola desde el hilo 0
Tiempo total de ejecución con 1 hilos: 0.000266 segundos
Ejecutando con 5 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Tiempo total de ejecución con 5 hilos: 0.000230 segundos
Ejecutando con 10 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Hola desde el hilo 5
Hola desde el hilo 6
Hola desde el hilo 7
Hola desde el hilo 8
Hola desde el hilo 9
Tiempo total de ejecución con 10 hilos: 0.000410 segundos
Finalizó la ejecución del programa principal
greCIA@grecia:~/Descargas/02.hilos$

```

```

Finalizó la ejecución del programa principal
greCIA@grecia:~/Descargas/02.hilos$ time ./hilos
Ejecutando con 1 hilo...
Hola desde el hilo 0
Tiempo total de ejecución con 1 hilos: 0.000273 segundos
Ejecutando con 5 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Tiempo total de ejecución con 5 hilos: 0.000149 segundos
Ejecutando con 10 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Hola desde el hilo 5
Hola desde el hilo 6
Hola desde el hilo 7
Hola desde el hilo 8
Hola desde el hilo 9
Tiempo total de ejecución con 10 hilos: 0.000270 segundos
Finalizó la ejecución del programa principal

real    0m0,004s
user    0m0,002s
sys     0m0,000s
greCIA@grecia:~/Descargas/02.hilos$

```

ANÁLISIS DE RESULTADOS

1. Comparación entre un programa multihilo y un monohilo

En la ejecución del programa se obtuvo un tiempo de 0.000273 segundos con 1 hilo, 0.000149 segundos con 5 hilos y 0.000270 segundos con 10 hilos. Se puede ver que con 5 hilos el tiempo fue menor. Sin embargo, al usar 10 hilos el tiempo volvió a aumentar. Esto indica que al usar más hilos se puede mejorar el rendimiento, pero tener una mayor cantidad de hilos no siempre significa que sea una ejecución más rápida.

2. Funciones utilizadas de la biblioteca pthread

Se utilizaron varias funciones de la biblioteca pthread. `pthread_create()` se usó para crear los hilos y `pthread_join()` para esperar a que cada hilo termine. También se utilizaron `pthread_mutex_lock()` y `pthread_mutex_unlock()` para controlar el acceso de los hilos a una parte del programa. Además, `pthread_mutex_init()` se utilizó para iniciar el mutex y `pthread_mutex_destroy()` para finalizarlo.

3. Comportamiento al aumentar el número de hilos

Al probar con diferentes cantidades de hilos, se obtuvo el menor tiempo con 5 hilos, con un resultado de 0.000149 segundos. Al usar 10 hilos, el tiempo aumentó a 0.000270 segundos. Esto puede ocurrir porque crear y controlar muchos hilos también necesita recursos del sistema, por lo que, aumentar el número de hilos no siempre mejora el rendimiento.

4. Condiciones de carrera o comportamientos inesperados

Durante el desarrollo de la práctica no se observaron problemas en la ejecución. El uso del mutex permitió controlar el acceso de los hilos y evitar que varios ejecutaran la misma parte al mismo tiempo.